

Java Exception Handling and Multithreading Documentation

1. Exception Handling:

Aim: To demonstrate exception handling using try, catch and finally.

Program:

```
public class ExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
            int[] x = {1, 2, 3};  
            System.out.println(x[5]);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Exception");  
        } finally {  
            System.out.println("Finally block executed");  
        }  
    }  
}
```

Output:

Arithmetic Exception Finally block executed.

Concept:

Handles runtime errors using try, catch and finally.

2. User-Defined Exception:

Aim: To demonstrate a user-defined exception in Java

Program:

```
class InvalidAgeException extends Exception {  
    InvalidAgeException(String msg) {  
        super(msg);  
    }  
}  
  
public class Voting {  
    public static void main(String[] args) {  
        int age = 16;  
        try {  
            if (age < 18)  
                throw new InvalidAgeException("Not eligible to vote");  
            System.out.println("Eligible to vote");  
        } catch (InvalidAgeException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Output:

Not eligible to vote

Concept:

Creates a custom exception using extends Exception and throw.

3. Multiple Exceptions:

Aim: To demonstrate handling multiple exceptions using multiple catch blocks.

Program:

```
public class MultipleExceptions {  
    public static void main(String[] args) {  
        try {  
            int a = 10 / 0;  
  
            int[] x = {1, 2};  
  
            System.out.println(x[5]);  
  
            int n = Integer.parseInt("abc");  
  
            System.out.println(n);  
        } catch (ArithmeticException e) {  
            System.out.println("Arithmetic Exception");  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index Exception");  
        } catch (NumberFormatException e) {  
            System.out.println("Number Format Exception");  
        }  
    }  
}
```

Output:

Arithmetic Exception.

Concept:

Uses multiple catch blocks to handle different errors.

4. Creating Threads:

Aim: To demonstrate creating threads using the Thread class.

Program:

```
class Numbers extends Thread {  
  
    public void run() {  
  
        for (int i = 1; i <= 10; i++)  
  
            System.out.println("Number: " + i);  
  
    }  
}  
  
class Squares extends Thread {  
  
    public void run() {  
  
        for (int i = 1; i <= 10; i++)  
  
            System.out.println("Square: " + (i * i));  
  
    }  
}  
  
public class ThreadDemo {  
  
    public static void main(String[] args) {  
  
        new Numbers().start();  
  
        new Squares().start();  
  
    }  
}
```

Output:

Numbers 1-10 and their squares are displayed. The order may vary because both threads execute concurrently.

Concept:

Creates threads by extending the Thread class.

5. Runnable Interface:

Aim: To demonstrate thread creation using the Runnable interface.

Program:

```
class MyThread implements Runnable {  
  
    String name;  
  
    int id;  
  
    MyThread(String name, int id) {  
  
        this.name = name;  
  
        this.id = id;  
  
    }  
  
    public void run() {  
  
        System.out.println("Name: " + name + ", ID: " + id);  
  
    }  
}  
  
public class RunnableDemo {  
  
    public static void main(String[] args) {  
  
        Thread t1 = new Thread(new MyThread("Thread-1", 101));  
  
        Thread t2 = new Thread(new MyThread("Thread-2", 102));  
  
        Thread t3 = new Thread(new MyThread("Thread-3", 103));  
  
  
        t1.start();  
  
        t2.start();  
  
        t3.start();  
  
    }  
}
```

```
}
```

Output:

Name and ID of all three threads are displayed.

Concept:

Creates threads by implementing the Runnable interface.

6. Thread Priority:

Aim: To demonstrate thread priority in Java.

Program:

```
class Task extends Thread {  
  
    Task(String name) {  
  
        super(name);  
  
    }  
  
    public void run() {  
  
        System.out.println(getName() + " Priority: " + getPriority());  
  
        System.out.println(getName() + " is running");  
  
    }  
}  
  
public class PriorityDemo {  
  
    public static void main(String[] args) {  
  
        Task t1 = new Task("Low");  
  
        Task t2 = new Task("Normal");  
  
        Task t3 = new Task("High");  
  
  
        t1.setPriority(Thread.MIN_PRIORITY);  
  
        t2.setPriority(Thread.NORM_PRIORITY);  
  
        t3.setPriority(Thread.MAX_PRIORITY);  
  
  
        t1.start();  
  
        t2.start();  
  
        t3.start();  
    }  
}
```

```
}
```

```
}
```

Output:

Low Priority: 1; Normal Priority: 5; High Priority: 10. Execution order is controlled by the JVM scheduler and is not guaranteed.

Concept:

Assigns priority using MIN, NORM and MAX priority.

7. Synchronization:

Aim: To demonstrate synchronization for shared resources.

Program:

```
class Bank {  
  
    int balance = 1000;  
  
    synchronized void withdraw(int amount) {  
  
        if (balance >= amount) {  
  
            System.out.println(Thread.currentThread().getName()  
                + " withdrawing " + amount);  
  
            balance -= amount;  
  
            System.out.println("Balance: " + balance);  
  
        } else {  
  
            System.out.println("Insufficient balance");  
  
        }  
  
    }  
  
}  
  
public class SyncDemo {  
  
    public static void main(String[] args) {  
  
        Bank b = new Bank();  
  
  
  
        Thread t1 = new Thread(() -> b.withdraw(700), "Thread-1");  
  
        Thread t2 = new Thread(() -> b.withdraw(700), "Thread-2");  
  
  
  
        t1.start();
```

```
        t2.start();  
    }  
}
```

Output:

One thread successfully withdraws 700; the other gets Insufficient balance. The synchronized method prevents unsafe simultaneous updates.

Concept:

Protects shared data using the synchronized keyword.

8. Inter-Thread Communication:

Aim: To demonstrate inter-thread communication using wait() and notify().

Program:

```
class Box {  
  
    int value;  
  
    boolean available = false;  
  
    synchronized void produce(int v) throws InterruptedException {  
  
        while (available)  
            wait();  
  
        value = v;  
        available = true;  
        System.out.println("Produced: " + value);  
        notify();  
    }  
  
    synchronized void consume() throws InterruptedException {  
  
        while (!available)  
            wait();  
  
        System.out.println("Consumed: " + value);  
        available = false;  
        notify();  
    }  
}
```

```

public class CommunicationDemo {

    public static void main(String[] args) {

        Box b = new Box();

        new Thread(() -> {

            try {

                for (int i = 1; i <= 5; i++)

                    b.produce(i);

            } catch (InterruptedException e) {

            }

        }).start();

        new Thread(() -> {

            try {

                for (int i = 1; i <= 5; i++)

                    b.consume();

            } catch (InterruptedException e) {

            }

        }).start();

    }

}

```

Output:

Produced: 1, Consumed: 1, Produced: 2, Consumed: 2, ... up to 5. Exact interleaving can vary.

Concept:

Uses wait() and notify() for thread communication.

9. Multiple Threads:

Aim: To demonstrate the execution of multiple threads concurrently.

Program:

```
class Numbers extends Thread {  
  
    public void run() {  
  
        for (int i = 1; i <= 5; i++)  
  
            System.out.println("Number: " + i);  
  
    }  
}
```

```
class Sum extends Thread {  
  
    public void run() {  
  
        int s = 0;  
  
        for (int i = 1; i <= 5; i++)  
  
            s += i;  
  
  
        System.out.println("Sum: " + s);  
  
    }  
}
```

```
class Even extends Thread {  
  
    public void run() {  
  
        for (int i = 1; i <= 10; i++)  
  
            if (i % 2 == 0)  
  
                System.out.println("Even: " + i);  
  
    }  
}
```

```
public class MultiThreadDemo {  
  
    public static void main(String[] args) {  
  
        new Numbers().start();  
  
        new Sum().start();  
  
        new Even().start();  
  
    }  
  
}
```

Output:

Numbers are displayed, Sum: 15 is displayed, and even numbers are displayed. Their order may vary.

Concept:

Runs multiple threads at the same time.

10. Thread Control:

Aim: To demonstrate thread control using sleep() and interrupt().

Program:

```
class Control extends Thread {  
    volatile boolean running = true;  
  
    public void run() {  
        while (running) {  
            System.out.println("Thread running...");  
            try {  
                Thread.sleep(500);  
            } catch (InterruptedException e) {  
                System.out.println("Thread interrupted");  
            }  
        }  
        System.out.println("Thread stopped");  
    }  
  
    void stopThread() {  
        running = false;  
        interrupt();  
    }  
}  
  
public class ControlDemo {  
    public static void main(String[] args) throws Exception {
```

```
Control t = new Control();  
  
t.start();  
  
Thread.sleep(1500);  
  
t.stopThread();  
  
}  
  
}
```

Output:

Thread running... is displayed several times, followed by Thread interrupted and Thread stopped.

Concept:

Controls thread execution using sleep() and interrupt().